



Network Security

Project report

Submitted by: MUHAMMAD AKASH WARIS
HABIB AHMED

Roll number: 23I-2110 , 23I-2078

Date: 29/11/2025

Table of Contents

Defense-in-Depth Network Security Architecture: A Multi-Layered Approach with Zeek, ClamAV, and Reverse Proxy	4
1.0 Abstract	4
2.0 Introduction & Problem Definition.....	4
2.1 Problem Statement	4
2.2 Complexity Analysis	4
3.0 System Design & Network Architecture.....	4
3.1 Network Zones & Addressing	5
3.2 Topology Diagram	5
4.0 Implementation Details	6
4.1 Phase 1: Perimeter Security (pfSense & Industry Standard Rules)	6
4.2 Phase 2: The Security Gateway (Reverse Proxy, WAF, & AV).....	7
4.3 Phase 3: Traffic Analysis Pipeline (Zeek & ELK).....	8
5.0 Testing, Evaluation, and Results	8
5.1 Vulnerability Testing (WAF)	8
5.2 Malware Simulation (AV Gateway)	9
5.3 Visibility Analysis (Kibana Dashboard)	10
6.0 VPN Services (Secure Remote Access).....	12
6.1 Implementation Details	12
6.2 Access Control & Firewalling.....	13
6.3 Challenges Faced & Solutions	13
6.4 Verification Results	13
7.0 Conclusion	15
8.0 References	15

Defense-in-Depth Network Security Architecture: A Multi-Layered Approach with Zeek, ClamAV, and Reverse Proxy

1.0 Abstract

This project addresses the critical lack of visibility and multi-layer protection in small-to-mid-scale enterprise networks. We designed and implemented a comprehensive "Defense-in-Depth" infrastructure using open-source technologies within a **GNS3** simulation environment. The architecture integrates **pfSense** for perimeter control with industry-standard filtering rules, **Zeek**, **ELK Stack (Elasticsearch, Logstash, Kibana)** for centralized threat visualization, and a custom **Reverse Proxy/Antivirus Gateway** using Nginx and ClamAV. The final system demonstrates practical network segmentation, real-time traffic monitoring, automated malware prevention, and the mitigation of web application attacks (SQLi/XSS).

2.0 Introduction & Problem Definition

2.1 Problem Statement

Modern enterprise networks face an increasing range of cyber threats targeting multiple layers—from perimeter defenses to endpoint systems. A simple firewall is no longer sufficient. Administrators often lack visibility into internal traffic and fail to stop application-layer attacks or malware propagation before it reaches the servers.

2.2 Complexity Analysis

This project qualifies as a "Complex Computing Problem" based on the following criteria:

Integration of Multiple Components: The solution requires the seamless interoperability of five distinct systems (pfSense, OVS, Zeek, ELK, Nginx/ClamAV) across different network zones within a GNS3.

Sophisticated Vulnerability Mitigation: It goes beyond static blocking by implementing dynamic WAF rules for SQL injection and behavioral analysis for anomaly detection.

Scalability and Automation: The system includes self-healing mechanisms (systemctl services) and scalable log pipelines to handle high-volume traffic data.

3.0 System Design & Network Architecture

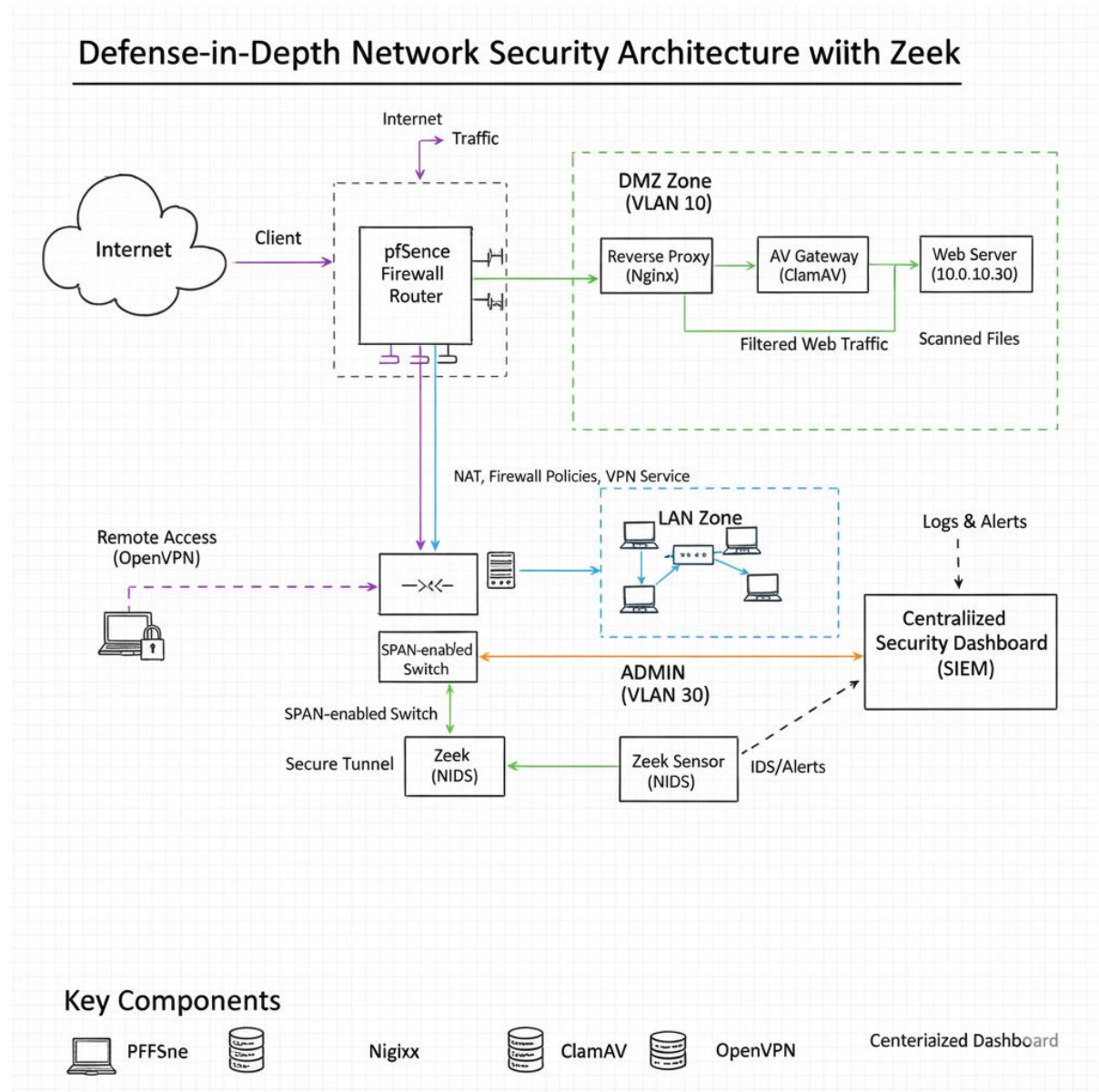
The network is segmented into four distinct security zones using **Open vSwitch (OVS)** to logically isolate traffic. We utilized a standard private addressing scheme to ensure scalability.

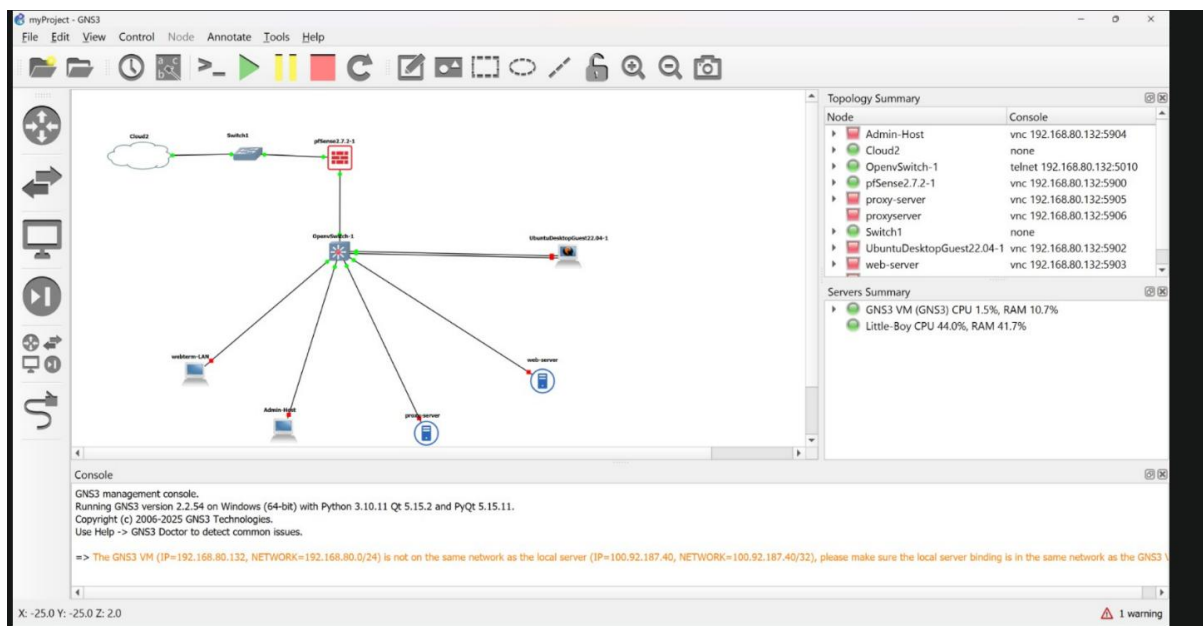
3.1 Network Zones & Addressing

Zone Name	VLAN ID	Network CIDR	Purpose
LAN Zone	Vlan 20	10.0.2.0/24	Internal employee workstations and general access.
Admin Zone	VLAN 10	10.0.1.0/24	Restricted management network for Zeek IDS and ELK Stack.
DMZ Zone	VLAN 30	10.0.3.0/24	Public-facing services (Reverse Proxy & Web Server).
Vpn zone	Vlan 40	10.0.4.0/24	Encrypted tunnel for vpn services

3.2 Topology Diagram

The diagram below illustrates the physical and logical connections, including the SPAN port configuration for traffic mirroring.





4.0 Implementation Details

The implementation was executed in distinct phases, moving from perimeter connectivity to advanced application security.

4.1 Phase 1: Perimeter Security (pfSense & Industry Standard Rules)

Key Firewall Policies Implemented:

Ingress Filtering: Default BLOCK on WAN. Only OpenVPN (UDP 1194) and Port 80 are allowed via NAT Port Forwarding to the DMZ Proxy (10.0.3.3). We also blocked "Bogon" networks on the WAN interface to prevent IP spoofing.

DMZ Isolation: A strict rule blocks traffic from DMZ to LAN. If the Web Server is hacked, the attacker cannot move laterally to the internal network (10.0.2.0/24).

Internal Segmentation: Both LAN and DMZ networks are explicitly blocked from accessing the sensitive Admin VLAN and the pfSense Web Dashboard (Port 443) to prevent unauthorized management access.

Reputation Filtering: High-priority malicious IPs are automatically blocked across all interfaces using the pfB_PRI1_v4 alias list to filter bad actors.

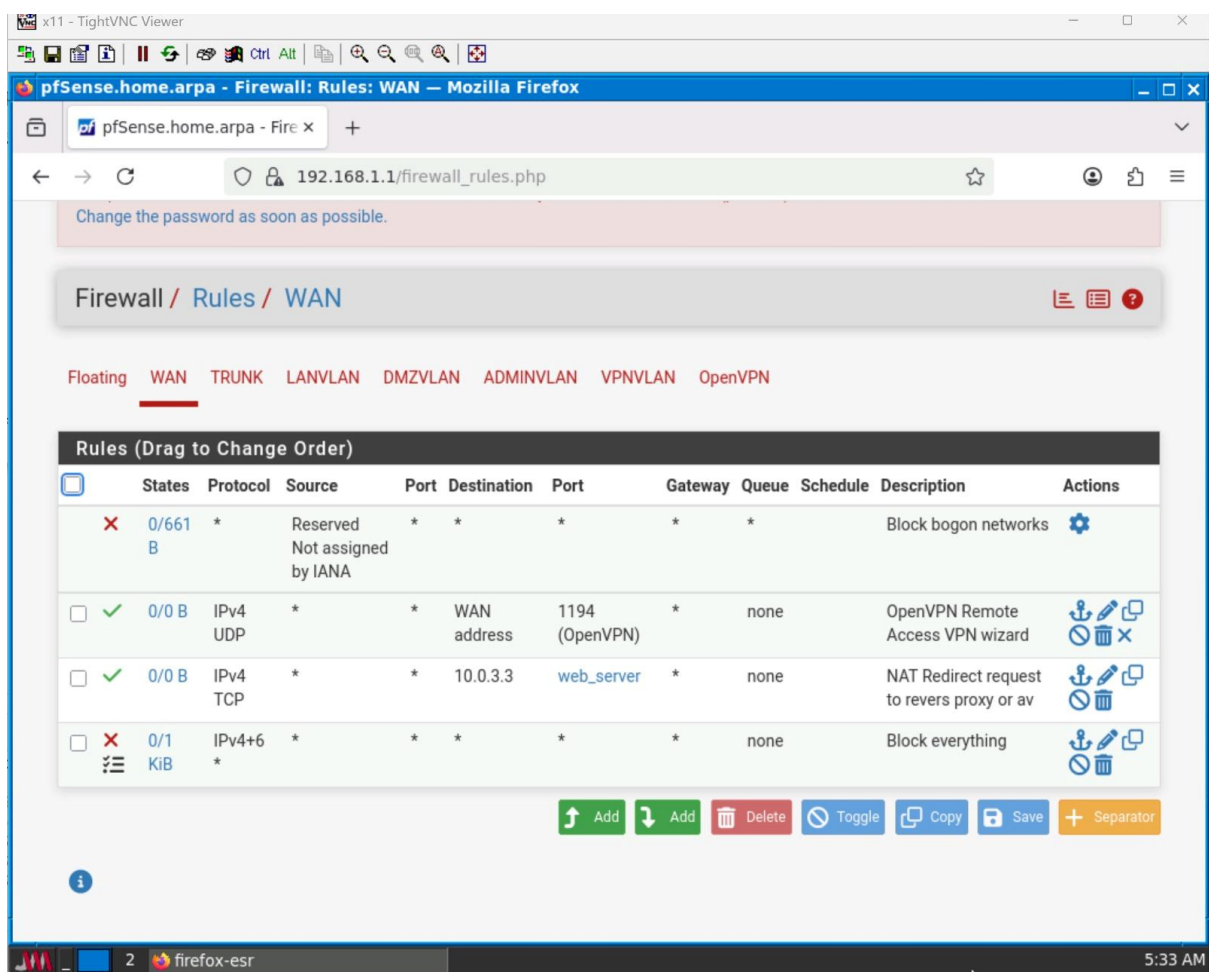


Figure : pfSense rules

4.2 Phase 2: The Security Gateway (Reverse Proxy, WAF, & AV)

To secure the web server (10.0.3.4), we transformed a single DMZ node (10.0.3.3) into a hardened Security Gateway using **Nginx** and **ClamAV**.

Web Application Firewall (WAF): We implemented a custom Nginx configuration with a `waf_rules.conf` include file. This uses Regex matching to inspect the `$request_uri` and block malicious patterns:

SQL Injection: Blocks `UNION SELECT`, `OR 1=1`, `DROP TABLE`.

XSS: Blocks `<script>`, `alert()`, and `javascript: handlers`.

Antivirus Integration: We developed a Python middleware (Flask) that sits on the proxy. It intercepts file uploads, scans them using `clamscan` via the `subprocess` module, and deletes any file matching a virus signature (e.g., EICAR) before it can reach the backend server.

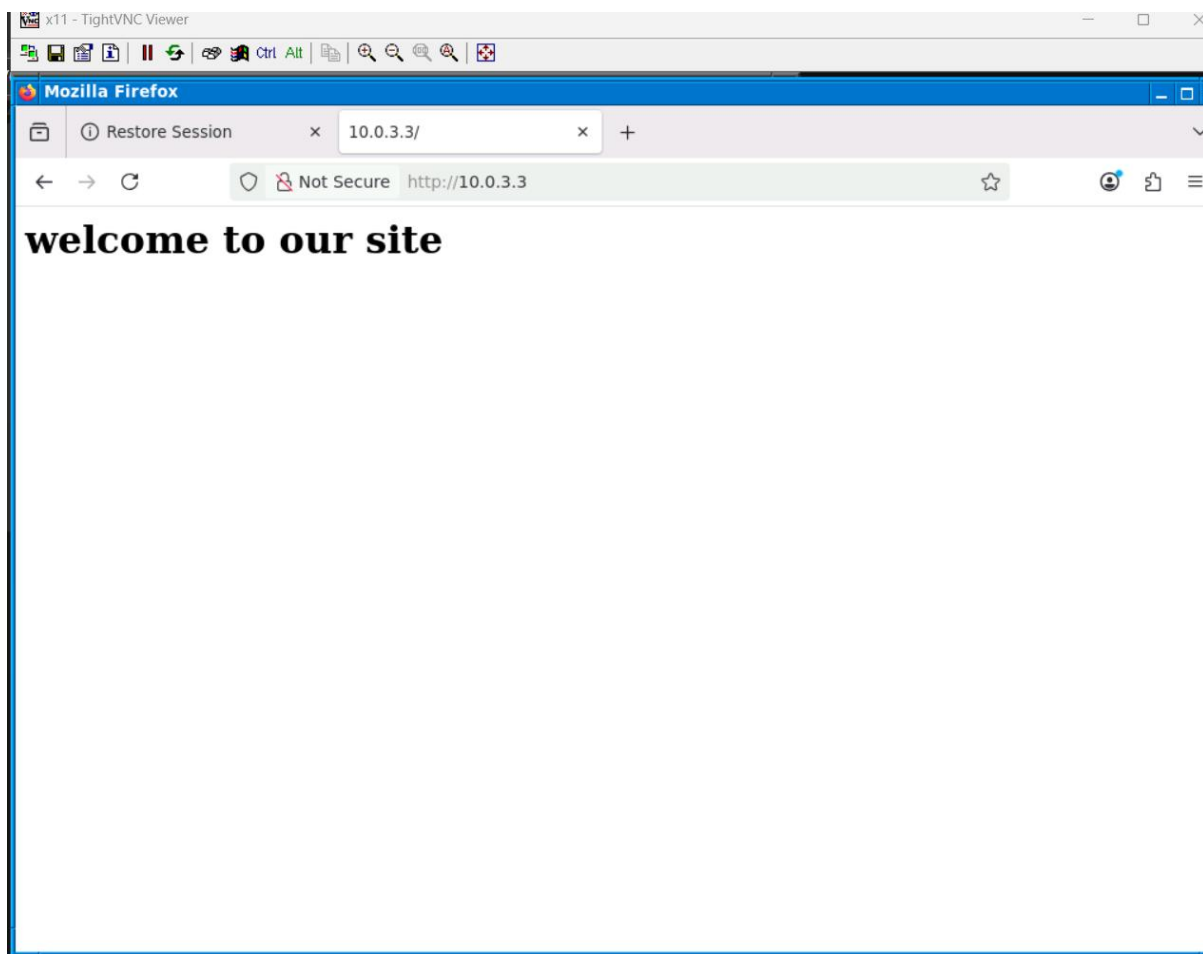


Figure : Successful Reverse Proxy redirection to the internal Web Server.

4.3 Phase 3: Traffic Analysis Pipeline (Zeek & ELK)

To achieve visibility, we built a multi-stage data pipeline residing in the Admin Network (10.0.1.0/24).

Zeek Configuration: Zeek was configured in a "cluster of one" mode to parse traffic from the mirrored interface (ens3) and output logs in JSON format.

Logstash Pipeline: A configuration (/etc/logstash/conf.d/zeek.conf) was created to ingest Zeek logs, parse the JSON, and index them into Elasticsearch.

Self-Healing Mechanism: To address Zeek service instability, a custom systemd service (/etc/systemd/system/zeek.service) was created to manage Zeek's lifecycle.

5.0 Testing, Evaluation, and Results

We validated the system using active attack simulations to prove the effectiveness of our rules.

5.1 Vulnerability Testing (WAF)

Test Case: Attempted an SQL Injection using ' OR 1=1 -- in the web search bar.

Result: Nginx immediately returned a **403 Forbidden** error. Access logs confirmed the WAF rule trigger.



Figure : Nginx WAF successfully blocking an SQL Injection attack.

5.2 Malware Simulation (AV Gateway)

Test Case: Uploaded the **EICAR** standard test file (`eicar.com.txt`) via the upload portal.

Result: The Security Gateway intercepted the file, deleted it, and displayed a "SECURITY ALERT: MALWARE DETECTED" page. The file never reached the backend server.

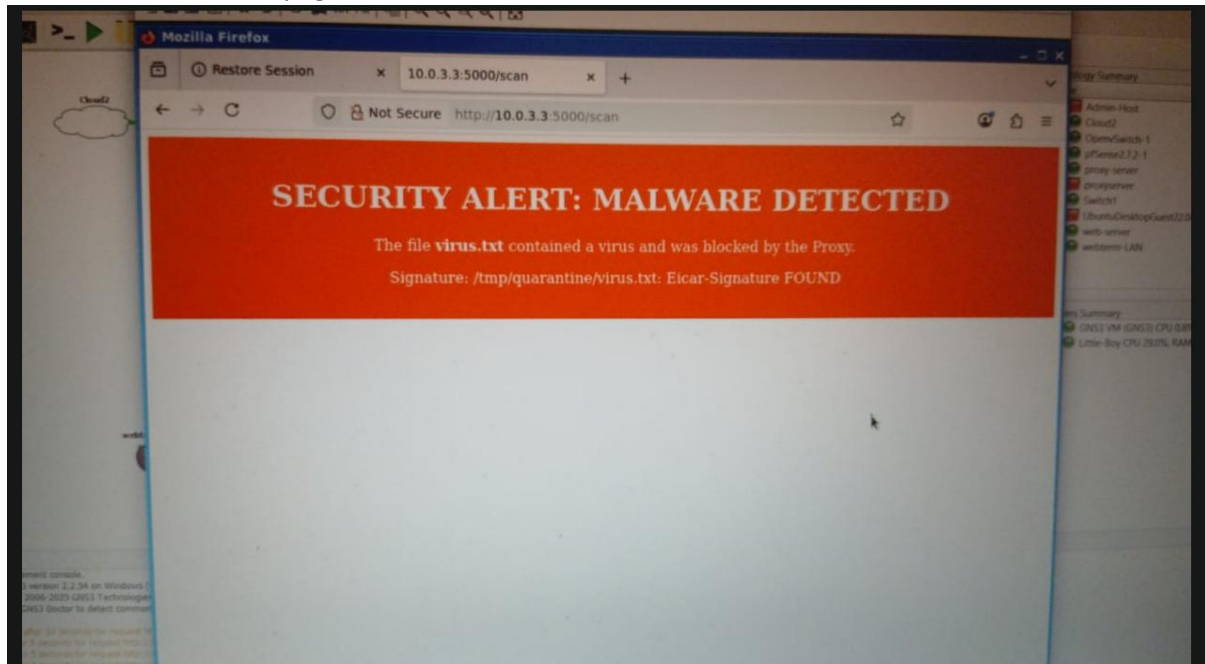


Figure : ClamAV Gateway intercepting a malicious file upload.

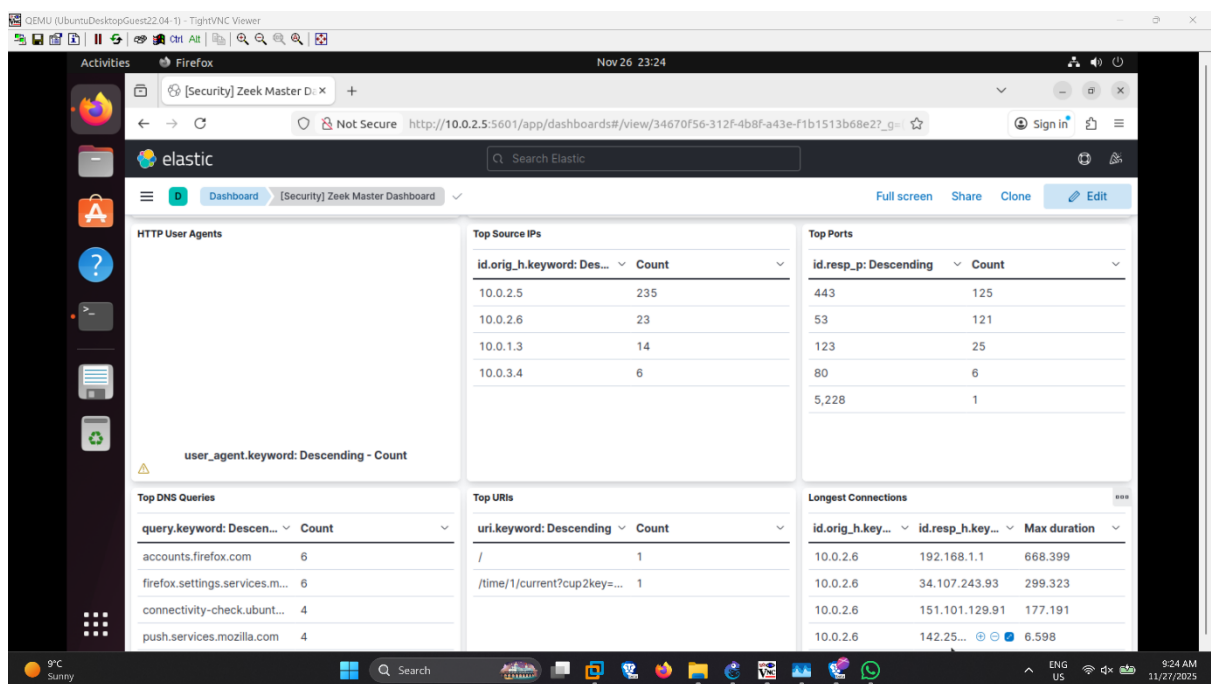
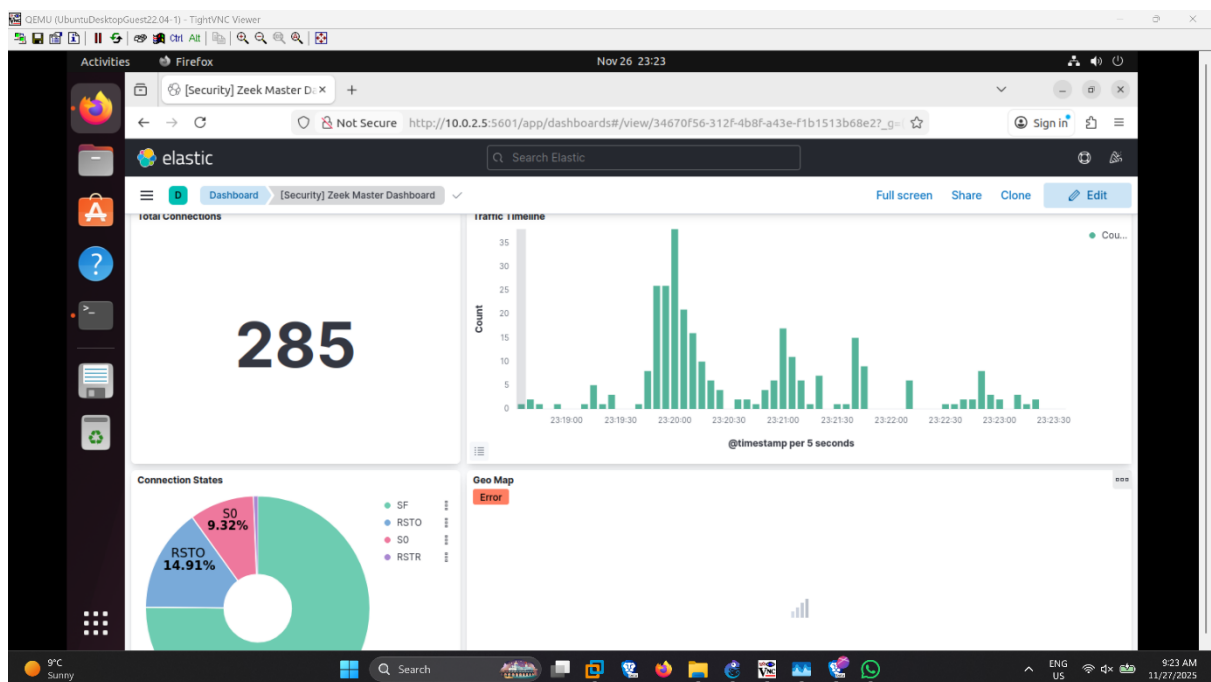
5.3 Visibility Analysis (Kibana Dashboard)

We utilized the ELK stack to visualize network traffic in real-time. The dashboard highlights potential threats and network usage.

Top Source IPs: Identifies the most active devices on the network.

Protocol Breakdown: Shows the distribution of HTTP, DNS, and TCP traffic.

Total connection state: Displays total connection states generated by Zeek's engine.



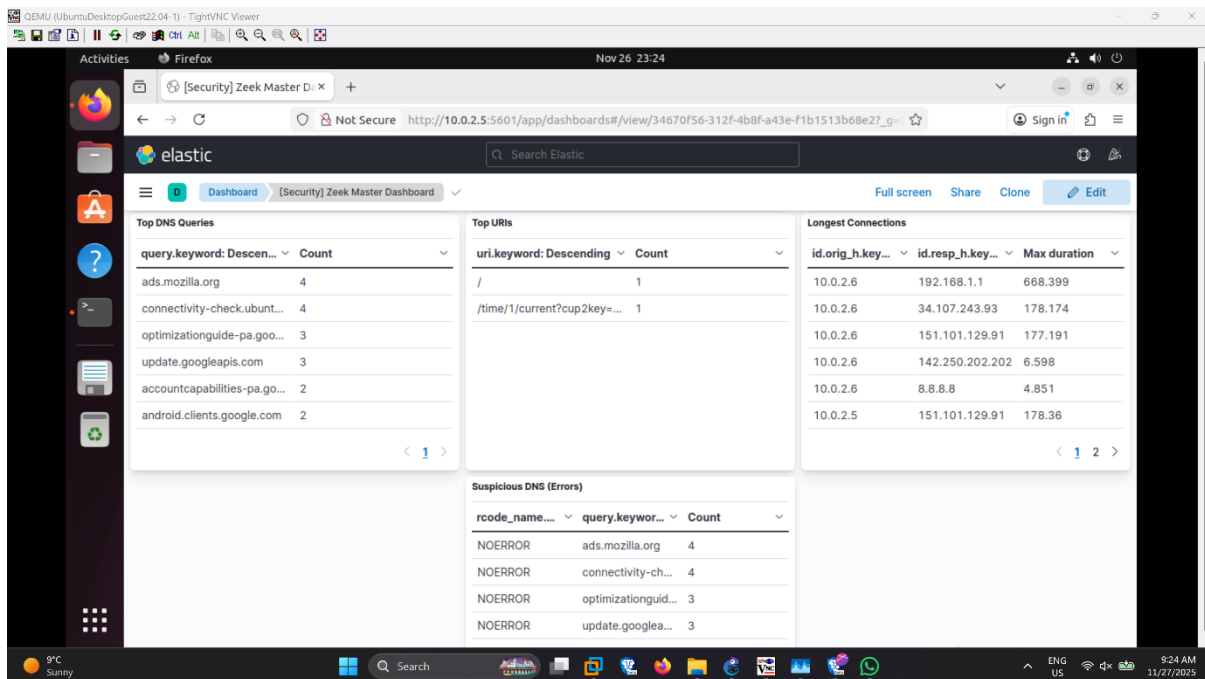


Figure : Kibana Security Dashboard visualizing real-time network traffic from Zeek.

6.0 VPN Services (Secure Remote Access)

To satisfy the requirement for secure telework and remote administration, we deployed an OpenVPN Remote Access Server. This allows authorized external users to securely access the internal LAN resources (10.0.2.0/24) over an encrypted tunnel, simulating a real-world "Work from Home" scenario.

6.1 Implementation Details

The VPN infrastructure was built using the **Public Key Infrastructure (PKI)** model on pfSense:

Certificate Authority (CA): We established an internal Root CA (MyProject_Root_CA) to sign digital certificates.

Authentication: Multi-factor authentication is enforced using **TLS Certificates** (something you have) plus **User Credentials** (something you know).

Tunnel Configuration: The server listens on **UDP Port 1194**. Upon connection, clients are assigned a virtual IP from the 10.0.4.0/24 pool.

Split Tunneling: We configured the server to push specific routes to the client, ensuring only traffic destined for the office LAN goes through the tunnel, while regular internet traffic bypasses it.

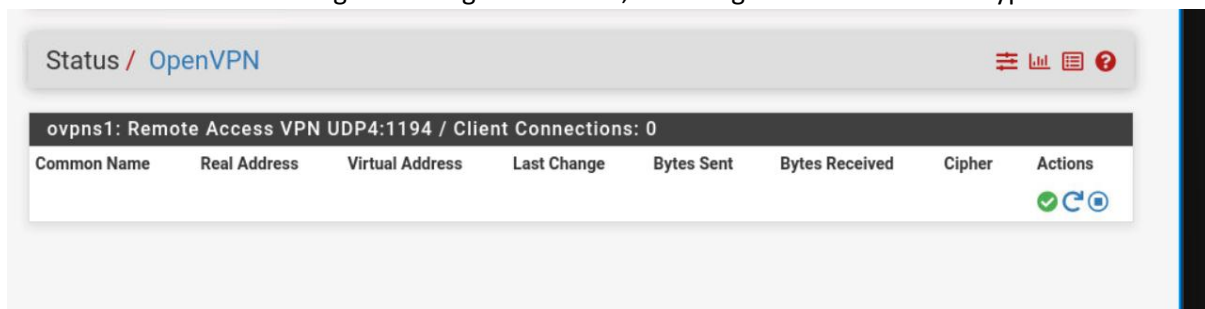


Figure : OpenVPN Server active and listening for incoming connections.

6.2 Access Control & Firewalling

Strict firewall rules were implemented to ensure the principle of Least Privilege:

WAN Rule: Allows encrypted UDP traffic on Port 1194 to enter the firewall.

OpenVPN Interface Rule: Allows traffic from authenticated VPN clients (10.0.4.x) to access the LAN net (10.0.2.x).

Restriction: VPN clients are strictly blocked from accessing the sensitive Admin Network (10.0.1.x) to prevent privilege escalation.

6.3 Challenges Faced & Solutions

During the implementation, we encountered significant routing conflicts that required advanced troubleshooting.

Issue 1: Interface Assignment & Routing Conflict

Problem: Although the VPN tunnel established successfully (clients received 10.0.4.2), pings to the internal Gateway (10.0.4.1) failed.

Root Cause: We had assigned the OpenVPN service to a specific pfSense Interface (VPN`VLAN`) which defaulted to "Static IPv4" configuration. This created a conflict where the kernel routing table fought with the OpenVPN routing logic.

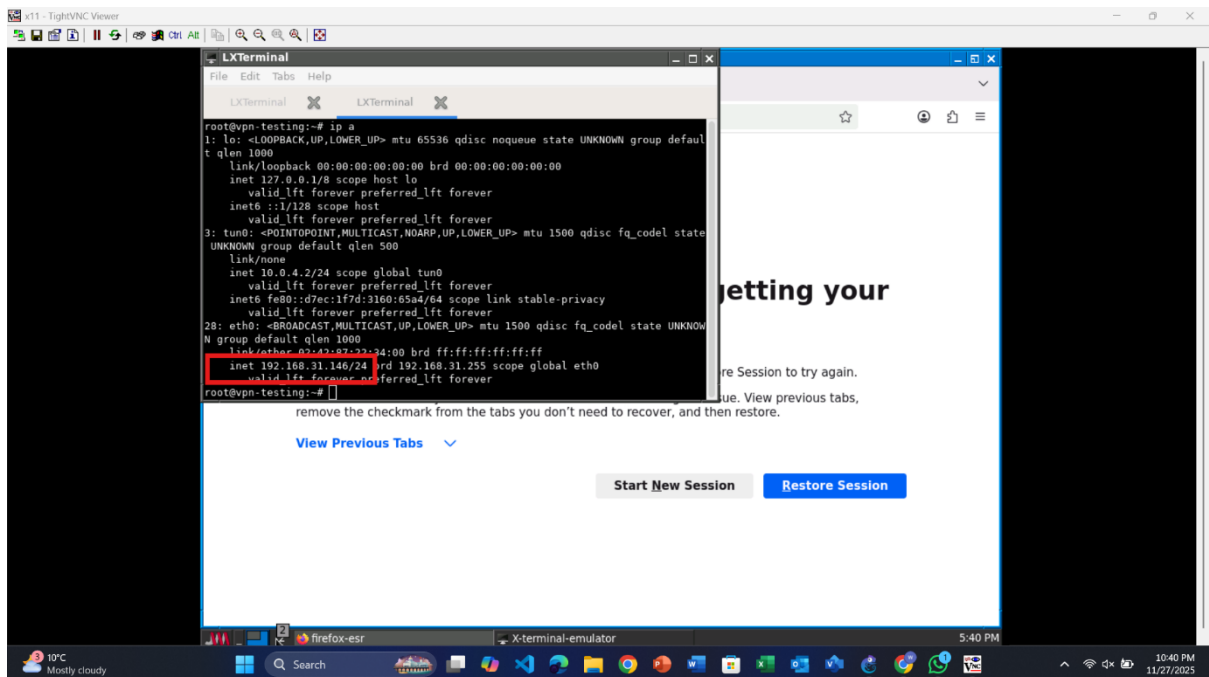
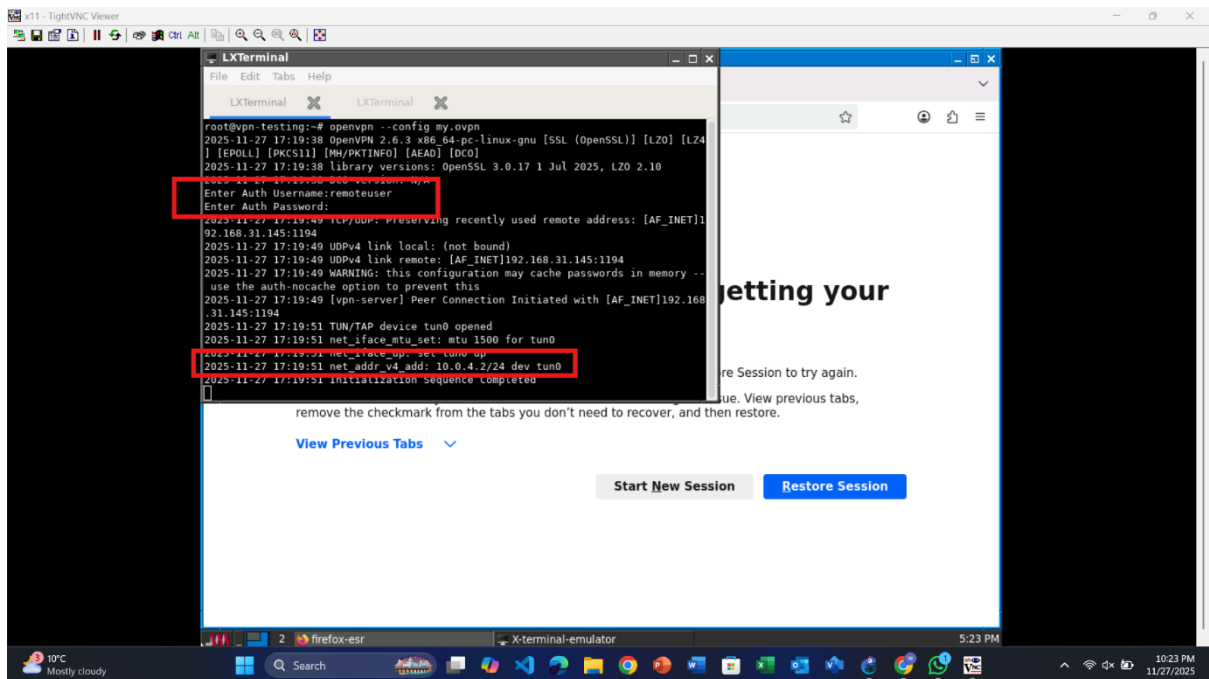
Solution: We modified the interface configuration type to "**None**". This allowed OpenVPN to manage the routing table exclusively, immediately resolving the connectivity issue and allowing ICMP traffic to pass to the LAN.

6.4 Verification Results

We verified the setup by connecting an external Linux client to the WAN network.

Log Confirmation: The client logs showed Initialization Sequence Completed and
net_addr_v4_add: 10.0.4.2.

Connectivity Test: From the external client, we successfully pinged the internal LAN Gateway (10.0.2.1), confirming that the encrypted tunnel bridges the external and internal networks effectively.



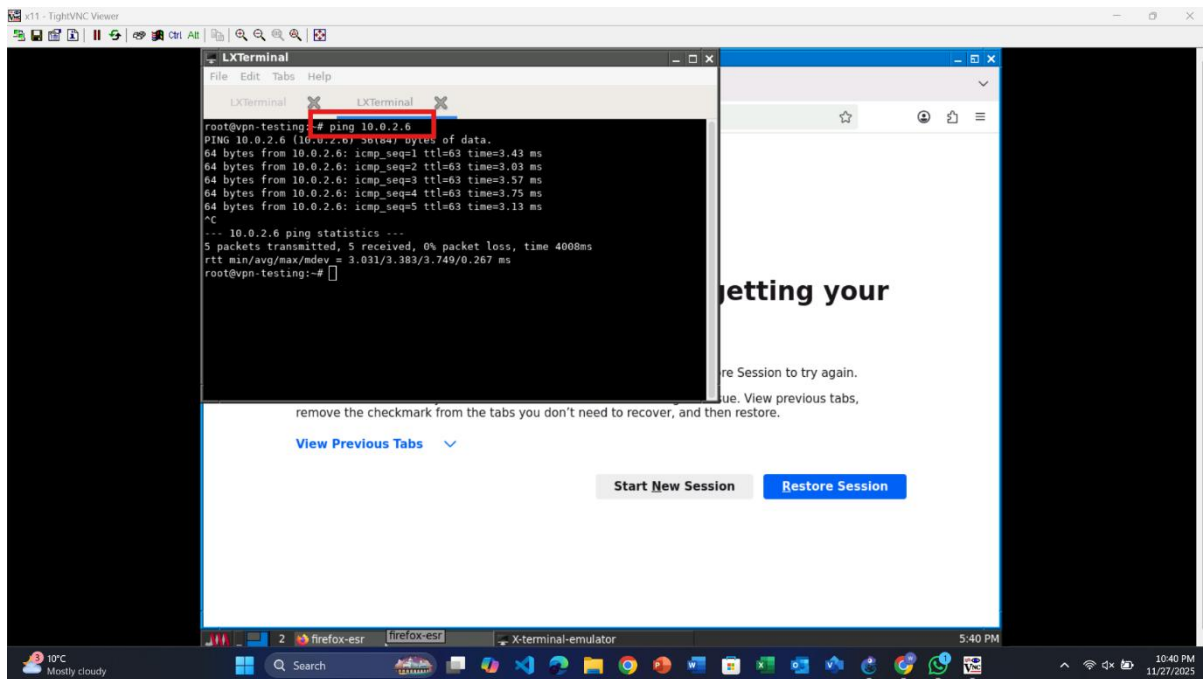


Figure : Successful connectivity test from an external VPN client to the internal LAN Gateway.

7.0 Conclusion

This project successfully demonstrates a robust, enterprise-grade security architecture. By combining the **Industry Standard Firewall Rules** of pfSense with the proactive blocking of **Nginx/ClamAV** and the deep visibility of **Zeek/ELK**, we have created a secure environment that effectively detects and mitigates complex threats. The system satisfies the core requirement of "Defense-in-Depth," providing redundancy and resilience against modern cyberattacks.

8.0 References

pfSense Documentation - Firewall Rule Best Practices.

Zeek Network Security Monitor Documentation.

Elastic Stack (ELK) Deployment Guide.

Nginx Reverse Proxy & WAF Configuration Guide.

ClamAV Antivirus Documentation.